SUPPLEMENTAL INFORMATION FOR SPARSITY-DRIVEN EMERGENCE OF FUNCTIONAL MODULARITY IN MULTITASK NEURAL NETWORKS

Bowen Shi[1,2] **Qi Ouyang**[1,3,4] **Long Qian**[2,3]
[1]Institute for Advanced Study in Physics, Zhejiang University, Hangzhou 310058, China
[2]Center for Quantitative Biology, Academy for Advanced Interdisciplinary Studies, Peking University, Beijing 100871, China
[3]ZJU–Hangzhou Global Scientific and Technological Innovation Center, Hangzhou 311200, China
[4]Institute of Fundamental and Transdisciplinary Research, Zhejiang University, Hangzhou, China
sbw1996@zju.edu.cn; qoy@zju.edu.cn; longqian@pku.edu.cn

October 21, 2026

S1 A Local Support-Recovery Argument for L_γ-Induced Subnetworks

This supplemental section gives a conditional proof explaining why sparsity-inducing L_γ regularization can make task-selective functional-token subnetworks separable by ablation or Taylor profiling. The argument starts from the linear multitask model

$$Y = XB_0 + E, \tag{S.1}$$

and then transfers the conclusion to a trained neural network through an explicit local-linear surrogate assumption. It should be read as a support-recovery theorem for the surrogate coordinates, not as a global theorem for nonlinear training dynamics.

S1.1 Linear surrogate and notation

Let n be the number of samples, p the number of candidate elementary coordinates, and R the number of tasks. In Eq. (S.1),

$$X \in \mathbb{R}^{n\times p}, \qquad Y = [Y_1, \dots, Y_R] \in \mathbb{R}^{n\times R}, \qquad B_0 = [\beta_{0,1}, \dots, \beta_{0,R}] \in \mathbb{R}^{p\times R}, \tag{S.2}$$

where $Y_r \in \mathbb{R}^n$ is the target vector for task r, $\beta_{0,r} \in \mathbb{R}^p$ is the true task-specific coefficient vector, and $E = [\varepsilon_1, \dots, \varepsilon_R]$ is the noise matrix. For each task,

$$Y_r = X\beta_{0,r} + \varepsilon_r. \tag{S.3}$$

We write

$$C = \frac{1}{n}X^\top X, \qquad \mathcal{S}_r = \{j : \beta_{0,jr} \neq 0\}, \qquad \mathcal{S}_r^c = \{1, \dots, p\} \setminus \mathcal{S}_r, \qquad s_r = |\mathcal{S}_r|. \tag{S.4}$$

Here $\mathcal{S}_r$ is the true support of task r. To avoid confusing support sets with importance scores, this section uses calligraphic $\mathcal{S}$ for supports and reserves symbols such as $S_{r,q}^{\mathrm{abl}}$ and $S_{r,q}^{\mathrm{tay}}$ for ablation and Taylor scores.

We estimate B_0 by

$$\widehat{B}_\gamma \in \arg\min_B Q_\gamma(B), \qquad Q_\gamma(B) = \frac{1}{2n}\|Y - XB\|_F^2 + \lambda \sum_{j=1}^{p}\sum_{r=1}^{R} |B_{jr}|^\gamma, \qquad 0 < \gamma \leq 1. \tag{S.5}$$

The objective is separable across task columns:

$$Q_\gamma(B) = \sum_{r=1}^{R} q_{\gamma,r}(\beta_r), \qquad q_{\gamma,r}(\beta) = \frac{1}{2n}\|Y_r - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|^\gamma. \tag{S.6}$$

It is therefore enough to prove the support claim for one fixed task. In the proof below we suppress the task index r, writing $Y, \beta_0, \varepsilon, \mathcal{S}$ for the corresponding single-task quantities.

S1.2 Assumptions

The following assumptions are standard support-recovery assumptions, with an additional local-curvature condition for the nonconvex case $0 < \gamma < 1$.

A1. Sparse and normalized design. The support size satisfies $s = |\mathcal{S}| \ll p$. The columns of X are normalized so that $C_{jj} = 1$, and $\|C\|_{op} \le M_C$ for a finite constant M_C. Here $\|\cdot\|_{op}$ denotes the spectral norm.

A2. Restricted identifiability. There are constants $m > 0$ and $M_S < \infty$ such that

$$\lambda_{\min}(C_{\mathcal{S}\mathcal{S}}) \ge m, \qquad \|C_{\mathcal{S}\mathcal{S}}^{-1}\|_\infty \le M_S. \tag{S.7}$$

For index sets A, B, C_{AB} denotes the submatrix of C with rows in A and columns in B. The matrix ∞-norm is the maximum absolute row sum.

A3. Noise correlation. The empirical noise-design correlation is uniformly bounded:

$$\left\| \frac{1}{n} X^\top \varepsilon \right\|_\infty \le \tau_n. \tag{S.8}$$

For normalized designs and sub-Gaussian noise, τ_n is typically of order $\sqrt{\log p / n}$ for one task, or $\sqrt{\log(pR)/n}$ uniformly across R tasks.

A4. Beta-min signal. The smallest active coefficient is bounded below:

$$b_{\min} = \min_{j \in \mathcal{S}} |\beta_{0,j}| > 0. \tag{S.9}$$

The tuning parameter λ is small enough that active coefficients do not cross zero. A convenient sufficient condition is

$$M_S\{\tau_n + \lambda\gamma(b_{\min}/2)^{\gamma-1}\} < b_{\min}/2. \tag{S.10}$$

For $\gamma = 1$, this is $M_S(\tau_n + \lambda) < b_{\min}/2$.

A5. Off-support incoherence for $\gamma = 1$. For the Lasso endpoint, assume the strict irrepresentable condition

$$\left\| C_{\mathcal{S}^c\mathcal{S}} C_{\mathcal{S}\mathcal{S}}^{-1} \operatorname{sign}(\beta_{0,\mathcal{S}}) \right\|_\infty \le 1 - \eta, \qquad 0 < \eta \le 1, \tag{S.11}$$

and assume the off-support noise is dominated by the Lasso penalty:

$$\left(1 + \left\| C_{\mathcal{S}^c\mathcal{S}} C_{\mathcal{S}\mathcal{S}}^{-1} \right\|_\infty\right) \tau_n \le \eta\lambda. \tag{S.12}$$

These two inequalities are used only for the convex $\gamma = 1$ support-recovery proof.

A6. Active-set curvature for $0 < \gamma < 1$. For $0 < \gamma < 1$, $|b|^\gamma$ is concave away from zero, so its second derivative is negative on active coordinates. We assume the restricted quadratic loss curvature dominates that negative curvature in the sign-preserving neighborhood $|b_j| \ge b_{\min}/2$: there is $\mu > 0$ such that

$$\lambda_{\min}(C_{\mathcal{S}\mathcal{S}}) - \lambda\gamma(1-\gamma)(b_{\min}/2)^{\gamma-2} \ge \mu. \tag{S.13}$$

In addition, the active gradient at the oracle coefficient is small enough that

$$\sqrt{s}\left\{\tau_n + \lambda\gamma(b_{\min}/2)^{\gamma-1}\right\} < \mu b_{\min}/4. \tag{S.14}$$

This condition ensures that the restricted nonconvex problem has an interior minimizer in the sign-preserving neighborhood.

S1.3 Support recovery for $0 < \gamma \le 1$

Proposition S1. Under A1–A6, the following statements hold for each task r.

For $\gamma = 1$, the convex Lasso problem has a global minimizer $\widehat{\beta}_{1,r}$ satisfying

$$\operatorname{supp}(\widehat{\beta}_{1,r}) = \mathcal{S}_r, \qquad \operatorname{sign}(\widehat{\beta}_{1,\mathcal{S}_r,r}) = \operatorname{sign}(\beta_{0,\mathcal{S}_r,r}). \tag{S.15}$$

For $0 < \gamma < 1$, the nonconvex objective has a strict local minimizer $\widehat{\beta}^{\mathrm{loc}}_{\gamma,r}$ satisfying

$$\operatorname{supp}(\widehat{\beta}^{\mathrm{loc}}_{\gamma,r}) = \mathcal{S}_r, \qquad \operatorname{sign}(\widehat{\beta}^{\mathrm{loc}}_{\gamma,\mathcal{S}_r,r}) = \operatorname{sign}(\beta_{0,\mathcal{S}_r,r}). \tag{S.16}$$

The second statement is local: it does not claim that all global minimizers have this support, nor that gradient descent from arbitrary initialization must reach this local minimizer.

Proof. Fix a task and omit the subscript r.

Case 1: $\gamma = 1$. The objective is convex. We use a primal-dual witness argument. The phrase Karush-Kuhn-Tucker (KKT) conditions refers here to the necessary and sufficient first-order optimality conditions for a convex optimization problem with a nonsmooth penalty, expressed through the subgradient of the ℓ_1 norm.

First solve the restricted Lasso problem on the true support:

$$\widehat{\beta}_{\mathcal{S}} \in \arg\min_{b\in\mathbb{R}^s} \left\{ \frac{1}{2n}\|Y - X_{\mathcal{S}} b\|_2^2 + \lambda\|b\|_1 \right\}, \qquad \widehat{\beta}_{\mathcal{S}^c} = 0. \tag{S.17}$$

On active coordinates, the KKT equation is

$$C_{\mathcal{S}\mathcal{S}}(\widehat{\beta}_{\mathcal{S}} - \beta_{0,\mathcal{S}}) - \frac{1}{n} X_{\mathcal{S}}^\top \varepsilon + \lambda z_{\mathcal{S}} = 0, \tag{S.18}$$

where $z_{\mathcal{S}} \in \partial\|\widehat{\beta}_{\mathcal{S}}\|_1$. If $\widehat{\beta}_{\mathcal{S}}$ has the correct sign, then $z_{\mathcal{S}} = \operatorname{sign}(\beta_{0,\mathcal{S}})$. Solving Eq. (S.18) gives

$$\widehat{\beta}_{\mathcal{S}} - \beta_{0,\mathcal{S}} = C_{\mathcal{S}\mathcal{S}}^{-1}\left(\frac{1}{n} X_{\mathcal{S}}^\top \varepsilon - \lambda z_{\mathcal{S}}\right). \tag{S.19}$$

By A2–A4 with $\gamma = 1$,

$$\|\widehat{\beta}_{\mathcal{S}} - \beta_{0,\mathcal{S}}\|_\infty \le M_{\mathcal{S}}(\tau_n + \lambda) < b_{\min}/2. \tag{S.20}$$

Thus every active coefficient remains nonzero and has the same sign as $\beta_{0,\mathcal{S}}$. This validates the chosen active subgradient.

It remains to verify the inactive KKT condition. For $j \in \mathcal{S}^c$, the subgradient condition for $\widehat{\beta}_j = 0$ is

$$\left\| C_{\mathcal{S}^c\mathcal{S}}(\widehat{\beta}_{\mathcal{S}} - \beta_{0,\mathcal{S}}) - \frac{1}{n} X_{\mathcal{S}^c}^\top \varepsilon \right\|_\infty \le \lambda. \tag{S.21}$$

Substituting the active solution with $z_{\mathcal{S}} = \operatorname{sign}(\beta_{0,\mathcal{S}})$, the left side of Eq. (S.21) is at most

$$\lambda \left\| C_{\mathcal{S}^c\mathcal{S}} C_{\mathcal{S}\mathcal{S}}^{-1} \operatorname{sign}(\beta_{0,\mathcal{S}}) \right\|_\infty + \left(1 + \left\| C_{\mathcal{S}^c\mathcal{S}} C_{\mathcal{S}\mathcal{S}}^{-1} \right\|_\infty\right) \tau_n. \tag{S.22}$$

By A5 this is bounded by $\lambda(1-\eta) + \eta\lambda = \lambda$. Therefore the full vector $(\widehat{\beta}_{\mathcal{S}}, 0)$ satisfies the KKT conditions. Since the $\gamma = 1$ objective is convex, these conditions certify a global minimizer with support $\mathcal{S}$ and correct signs.

Case 2: $0 < \gamma < 1$. The penalty is nonconvex and non-Lipschitz at zero, so KKT conditions at zero are not the right tool. We instead prove the existence of an oracle-supported strict local minimizer.

Consider the restricted objective on the true support:

$$q_{\gamma,\mathcal{S}}(b) = \frac{1}{2n}\|Y - X_{\mathcal{S}} b\|_2^2 + \lambda \sum_{j\in\mathcal{S}} |b_j|^\gamma. \tag{S.23}$$

Define the sign-preserving neighborhood

$$\mathcal{N}_{\mathcal{S}} = \{b : \|b - \beta_{0,\mathcal{S}}\|_\infty \le b_{\min}/2\}. \tag{S.24}$$

Inside this neighborhood all active coordinates satisfy $|b_j| \ge b_{\min}/2$. For any point in $\mathcal{N}_{\mathcal{S}}$, the Hessian of the restricted objective is

$$\nabla^2 q_{\gamma,\mathcal{S}}(b) = C_{\mathcal{S}\mathcal{S}} - \lambda\gamma(1-\gamma)\operatorname{diag}(|b_j|^{\gamma-2} : j \in \mathcal{S}), \tag{S.25}$$

which is bounded below by μI by A6. Hence $q_{\gamma,\mathcal{S}}$ is μ-strongly convex on $\mathcal{N}_{\mathcal{S}}$, even though the full objective is nonconvex outside this neighborhood.

At the oracle coefficient,

$$g_0 = \nabla q_{\gamma,\mathcal{S}}(\beta_{0,\mathcal{S}}) = -\frac{1}{n} X_{\mathcal{S}}^\top \varepsilon + \lambda\gamma|\beta_{0,\mathcal{S}}|^{\gamma-1} \operatorname{sign}(\beta_{0,\mathcal{S}}). \tag{S.26}$$

By A3 and A6,

$$\|g_0\|_2 \le \sqrt{s}\{\tau_n + \lambda\gamma(b_{\min}/2)^{\gamma-1}\} < \mu b_{\min}/4. \tag{S.27}$$

For any boundary point $b = \beta_{0,\mathcal{S}} + u$ of $\mathcal{N}_{\mathcal{S}}$, $\|u\|_2 \ge b_{\min}/2$. Strong convexity gives

$$q_{\gamma,\mathcal{S}}(\beta_{0,\mathcal{S}} + u) - q_{\gamma,\mathcal{S}}(\beta_{0,\mathcal{S}}) \ge u^\top g_0 + \frac{\mu}{2}\|u\|_2^2 \ge -\|u\|_2\|g_0\|_2 + \frac{\mu}{2}\|u\|_2^2 > 0. \tag{S.28}$$

Therefore the minimizer of $q_{\gamma,\mathcal{S}}$ over the compact set $\mathcal{N}_\mathcal{S}$ lies in its interior. Denote it by $\widehat{\beta}_\mathcal{S}$. It satisfies the active first-order equation

$$C_{\mathcal{S}\mathcal{S}}(\widehat{\beta}_\mathcal{S} - \beta_{0,\mathcal{S}}) - \frac{1}{n} X_\mathcal{S}^\top \varepsilon + \lambda\gamma |\widehat{\beta}_\mathcal{S}|^{\gamma-1} \operatorname{sign}(\widehat{\beta}_\mathcal{S}) = 0. \tag{S.29}$$

Because $\widehat{\beta}_\mathcal{S} \in \mathcal{N}_\mathcal{S}$, all active signs are preserved.

We now show that $(\widehat{\beta}_\mathcal{S}, 0)$ is a strict local minimizer in the full p-dimensional space. Perturb the active and inactive coordinates by $u \in \mathbb{R}^s$ and $v \in \mathbb{R}^{p-s}$, respectively. A Taylor expansion around $(\widehat{\beta}_\mathcal{S}, 0)$, using Eq. (S.29) for the active coordinates, yields

$$\Delta q(u,v) = q_\gamma(\widehat{\beta}_\mathcal{S} + u, v) - q_\gamma(\widehat{\beta}_\mathcal{S}, 0) \tag{S.30}$$

$$\geq \frac{\mu}{2}\|u\|_2^2 + u^\top C_{\mathcal{S}\mathcal{S}^c} v + a^\top v - \frac{M_C}{2}\|v\|_2^2 + \lambda \sum_{j \in \mathcal{S}^c} |v_j|^\gamma, \tag{S.31}$$

where

$$a = C_{\mathcal{S}^c\mathcal{S}}(\widehat{\beta}_\mathcal{S} - \beta_{0,\mathcal{S}}) - \frac{1}{n} X_{\mathcal{S}^c}^\top \varepsilon \tag{S.32}$$

is finite by A1–A4. For $0 < \gamma < 1$, the term $\sum_j |v_j|^\gamma$ dominates both $\|v\|_1$ and $\|v\|_2^2$ as $\|v\|_\infty \to 0$: for sufficiently small $\|v\|_\infty$,

$$\|v\|_1 \leq \|v\|_\infty^{1-\gamma} \sum_j |v_j|^\gamma, \qquad \|v\|_2^2 \leq \|v\|_\infty^{2-\gamma} \sum_j |v_j|^\gamma. \tag{S.33}$$

The cross term satisfies, for any $\alpha > 0$,

$$|u^\top C_{\mathcal{S}\mathcal{S}^c} v| \leq \alpha \|u\|_2^2 + \frac{\|C_{\mathcal{S}\mathcal{S}^c}\|_{op}^2}{4\alpha} \|v\|_2^2. \tag{S.34}$$

Choose $\alpha < \mu/4$, then choose the neighborhood in v small enough that the inactive singular penalty dominates the finite linear term $a^\top v$, the quadratic term in v, and the remaining cross-term bound. Eq. (S.31) is then strictly positive for every nonzero sufficiently small perturbation (u, v). Thus $(\widehat{\beta}_\mathcal{S}, 0)$ is a strict local minimizer with support $\mathcal{S}$. This proves the nonconvex local statement.

Repeating the single-task argument for each task column proves the multitask statement, because Eq. (S.5) separates across tasks.

S1.4 From support recovery to score-separable functional tokens

Support recovery says that the effective coordinates of task r form a set $\mathcal{S}_r$. The experiments, however, do not observe $\beta_{0,r}$ directly. They estimate functional supports through ablation and Taylor scores. This subsection proves the link between the true support $\mathcal{S}_r$, the ablation score $S_{r,q}^{\mathrm{abl}}$, and the Taylor score $S_{r,q}^{\mathrm{tay}}$.

Let f_θ be a trained network and fix a local coordinate block ℓ. Let $z_q(x)$ denote the scalar local coordinate associated with weight token q, for example the coordinate appearing in a parameter-Jacobian linearization around the trained parameters. Let $f_r(x) \in \mathbb{R}^d$ be the output segment for task r, where $d = P$ in the experiments. Assume the following local-linear surrogate:

$$f_r(x) = \sum_{q=1}^{d_\ell} z_q(x) b_{qr} + \xi_r(x), \qquad b_{qr} \in \mathbb{R}^d, \tag{S.35}$$

where $\xi_r(x)$ is a remainder term. The task support in this layer is

$$\mathcal{S}_{\ell,r} = \{q : \|b_{qr}\|_2 > 0\}. \tag{S.36}$$

Define the ideal coefficient-energy score

$$G_{r,q} = \mathbb{E}\left[z_q(x)^2\right] \|b_{qr}\|_2^2. \tag{S.37}$$

Assume an active margin: there exists $\kappa > 0$ such that

$$q \in \mathcal{S}_{\ell,r} \quad \Rightarrow \quad G_{r,q} \geq \kappa, \qquad q \notin \mathcal{S}_{\ell,r} \quad \Rightarrow \quad G_{r,q} = 0. \tag{S.38}$$

This margin excludes unidentifiable active coordinates with zero coordinate variance or vanishingly small effective coefficients.

Ablation score. Let $f_r^{(-q)}(x)$ be the output after removing token q. The population ablation score is

$$S_{r,q}^{\mathrm{abl}} = \mathbb{E}\left\| f_r(x) - f_r^{(-q)}(x) \right\|_2^2. \tag{S.39}$$

If removing q changes only the corresponding local term up to a remainder

$$e_{r,q}^{\mathrm{abl}}(x) = f_r(x) - f_r^{(-q)}(x) - z_q(x) b_{qr}, \qquad \mathbb{E}\|e_{r,q}^{\mathrm{abl}}(x)\|_2^2 \le \epsilon_{\mathrm{abl}}^2, \tag{S.40}$$

and if $G_{r,q} \le M_G$ uniformly, then

$$\left| S_{r,q}^{\mathrm{abl}} - G_{r,q} \right| = \left| \mathbb{E}\|z_q b_{qr} + e_{r,q}^{\mathrm{abl}}\|_2^2 - \mathbb{E}\|z_q b_{qr}\|_2^2 \right| \tag{S.41}$$

$$\le 2\sqrt{M_G}\epsilon_{\mathrm{abl}} + \epsilon_{\mathrm{abl}}^2 =: \delta_{\mathrm{abl}}. \tag{S.42}$$

Thus ablation score is a controlled perturbation of the ideal support energy $G_{r,q}$.

Taylor score. Introduce a multiplicative gate m_q on token q, so that the local surrogate becomes

$$f_r(x; m) = \sum_{k=1}^{d_\ell} m_k z_k(x) b_{kr} + \xi_r(x; m), \qquad m_k = 1 \text{ at the trained network.} \tag{S.43}$$

The population first-order Taylor score is

$$S_{r,q}^{\mathrm{tay}} = \mathbb{E}\left\| \left. \frac{\partial f_r(x; m)}{\partial m_q} \right|_{m=\mathbf{1}} \right\|_2^2. \tag{S.44}$$

Because

$$\left. \frac{\partial f_r(x; m)}{\partial m_q} \right|_{m=\mathbf{1}} = z_q(x) b_{qr} + e_{r,q}^{\mathrm{tay}}(x), \qquad e_{r,q}^{\mathrm{tay}}(x) = \left. \frac{\partial \xi_r(x; m)}{\partial m_q} \right|_{m=\mathbf{1}}, \tag{S.45}$$

the same calculation gives, if $\mathbb{E}\|e_{r,q}^{\mathrm{tay}}(x)\|_2^2 \le \epsilon_{\mathrm{tay}}^2$,

$$\left| S_{r,q}^{\mathrm{tay}} - G_{r,q} \right| \le 2\sqrt{M_G}\epsilon_{\mathrm{tay}} + \epsilon_{\mathrm{tay}}^2 =: \delta_{\mathrm{tay}}. \tag{S.46}$$

Score-support relation. Define score-threshold support estimates

$$\widehat{\mathcal{S}}_{\ell,r}^{\mathrm{abl}}(t) = \{q : S_{r,q}^{\mathrm{abl}} > t\}, \qquad \widehat{\mathcal{S}}_{\ell,r}^{\mathrm{tay}}(t) = \{q : S_{r,q}^{\mathrm{tay}} > t\}. \tag{S.47}$$

Let $\delta = \max\{\delta_{\mathrm{abl}}, \delta_{\mathrm{tay}}\}$. If $\delta < \kappa/2$, then for every threshold

$$t \in (\delta, \kappa - \delta) \tag{S.48}$$

we have

$$\widehat{\mathcal{S}}_{\ell,r}^{\mathrm{abl}}(t) = \widehat{\mathcal{S}}_{\ell,r}^{\mathrm{tay}}(t) = \mathcal{S}_{\ell,r}. \tag{S.49}$$

Indeed, if $q \notin \mathcal{S}_{\ell,r}$, then $G_{r,q} = 0$, so both scores are at most $\delta < t$. If $q \in \mathcal{S}_{\ell,r}$, then $G_{r,q} \ge \kappa$, so both scores are at least $\kappa - \delta > t$. Moreover,

$$\left| S_{r,q}^{\mathrm{abl}} - S_{r,q}^{\mathrm{tay}} \right| \le \delta_{\mathrm{abl}} + \delta_{\mathrm{tay}}, \tag{S.50}$$

so ablation and Taylor are close because both estimate the same ideal energy $G_{r,q}$, not because they are algebraically identical interventions.

The empirical scores have finite-sample error and, for Taylor profiling, random-probe error. If uniform bounds of sizes c_N and c_K hold for these two errors, the same conclusion holds after replacing δ by $\delta + c_N + c_K$. Four probes do not automatically guarantee the required bound; no numerical probe-error guarantee is asserted here. This is the formal relation between the recovered task support $\mathcal{S}_{\ell,r}$, the ablation score $S_{r,q}^{\mathrm{abl}}$, and the Taylor score $S_{r,q}^{\mathrm{tay}}$.

S1.5 From linear supports to functional-token subnetworks

The support-recovery result applies directly to coordinates on which the regularized objective acts, such as readout coefficients, explicit gates, path coefficients, or parameter-Jacobian coordinates. For scalar weight coordinates in a nonlinear network, raw sparsity must still be interpreted through a local surrogate. The bridge from Eq. (S.1) to a neural network therefore requires a compatibility assumption: in the chosen local coordinates, the trained network is well approximated by Eq. (S.35), and the effect of L_γ regularization is reflected in sparse, stable effective coefficients b_{qr} with a nonzero score margin κ.

The support-recovery theorem and its singularity argument for $0 < \gamma < 1$ concern the idealized, unshifted penalty $|\beta|^\gamma$. The neural-network implementation instead uses $(|\theta| + 10^{-8})^\gamma$ with per-matrix normalization (Section S2). For this shifted penalty, the one-sided derivative has finite magnitude at zero; the unshifted singularity argument therefore does not directly prove local stability for the implemented objective. The theoretical connection remains conditional on the local-surrogate and compatibility assumptions above.

The support-recovery statement applies only when the realized score threshold, including the score floor, lies inside the assumed separation gap. A fixed 75th-percentile rule does not by itself guarantee this condition or identify the true support size. The theoretical noise bound τ_n is distinct from the extraction quantile $\tau = 0.75$.

Under these local-surrogate, compatibility and score-separation assumptions, Proposition S1 yields task-specific supports $\mathcal{S}_{\ell,r}$. Eq. (S.49) then shows that ablation and Taylor profiling recover those supports by thresholding. The task signature of token q is

$$\mathcal{R}_q = \{r : q \in \mathcal{S}_{\ell,r}\}. \tag{S.51}$$

For a task set $c \subseteq \{1, \ldots, R\}$, the retained functional-token subnetwork is the support union

$$U_c = \bigcup_{r \in c} \mathcal{S}_{\ell,r} = \{q : \mathcal{R}_q \cap c \neq \varnothing\}. \tag{S.52}$$

If $r \in c$, then $\mathcal{S}_{\ell,r} \subseteq U_c$. Provided the local approximation remains valid after the simultaneous mask is applied to all analyzed matrices, the surrogate gives

$$f_r^{(U_c)}(x) \approx \sum_{q \in \mathcal{S}_{\ell,r}} z_q(x) b_{qr} \approx f_r(x), \tag{S.53}$$

which is a conditional output-preservation statement, not a general sufficiency guarantee for a nonlinear network. Simultaneous removal of many weights can change activations and invalidate a single-coordinate approximation. Preserving thresholded predictions additionally requires the joint-mask output error to remain smaller than the original distance of each output from its decision threshold (0.5 for binary outputs). Support inclusion alone does not establish this stability or margin condition. If $s \notin c$, and the same approximation remains valid, then

$$f_s^{(U_c)}(x) \approx \sum_{q \in U_c \cap \mathcal{S}_{\ell,s}} z_q(x) b_{qs}. \tag{S.54}$$

A small intersection $U_c \cap \mathcal{S}_{\ell,s}$ alone does not imply reduced off-target accuracy: the magnitudes of omitted contributions, biases, approximation error and classification margins also matter. An accuracy decrease requires output changes that cross decision boundaries in directions that turn correct predictions into errors, with a net loss over any corrected errors. Sufficiency and specificity of the actual joint masks are therefore assessed empirically. Finally,

$$U_{c_1 \cup c_2} = U_{c_1} \cup U_{c_2}, \tag{S.55}$$

which explains why functional-token combinations can be composed by union of their recovered supports. This is the mathematical counterpart of the experimental masking rule: retain scalar weight tokens important for any selected task and remove the rest.

S1.6 Why the endpoint $\gamma = 1$ is included and why $\gamma > 1$ differs

The endpoint $\gamma = 1$ is included because the Lasso penalty has a subgradient interval at zero:

$$\partial |b| \big|_{b=0} = [-1, 1]. \tag{S.56}$$

This interval is exactly what allows the inactive KKT condition in Eq. (S.21) to hold with $\widehat{\beta}_j = 0$. Thus $\gamma = 1$ can produce exact zeros and support recovery under the irrepresentable and beta-min conditions above.

For $0 < \gamma < 1$, the derivative

$$\frac{d}{db} |b|^\gamma = \gamma |b|^{\gamma - 1} \operatorname{sign}(b), \qquad b \neq 0, \tag{S.57}$$

is singular as $b \to 0$. This singularity makes zero locally stable against sufficiently small inactive perturbations, which is why the nonconvex proof uses local domination by $\sum_j |v_j|^\gamma$ rather than a convex KKT certificate.

For $\gamma > 1$, neither mechanism is present. There is no Lasso subgradient interval at zero, and the derivative is finite at zero; for $\gamma = 2$, it is exactly zero at zero. Small empirical correlations can then be represented by small nonzero coefficients instead of being thresholded to exact zeros. Consequently, task supports need not have a stable active/inactive gap, and ablation or Taylor score distributions may not admit a clean threshold separating functional-token subnetworks.

S1.7 Conclusion of the argument

Combining the pieces gives the conditional chain

$$\begin{aligned} L_\gamma\ (0<\gamma\le 1) \quad &\Longrightarrow \quad \text{support recovery in a compatible local surrogate} \\ &\Longrightarrow \quad \mathcal{S}_{\ell,r} \quad \Longrightarrow \quad \widehat{\mathcal{S}}^{\mathrm{abl}}_{\ell,r} = \widehat{\mathcal{S}}^{\mathrm{tay}}_{\ell,r} = \mathcal{S}_{\ell,r}, \end{aligned} \tag{S.58}$$

provided the restricted-design, beta-min, noise, curvature, local-linearization, and score-margin assumptions hold. This proves the claimed relationship between L_γ-induced sparse supports and threshold-separable ablation/Taylor functional-token subnetworks, while making explicit the assumptions under which the conclusion is valid.

S2 Model Architectures and Training Details

S2.1 Multitask MLP

The robustness simulations use a fully connected Torch MLP with three hidden layers, each containing 300 ReLU units. The input dimension is $d_{\mathrm{in}} = 2P = 14$ for the binary arithmetic and logic tasks, whereas paired MNIST images are flattened and concatenated to give $d_{\mathrm{in}} = 2 \times 28 \times 28 = 1568$. The output dimension is $d_{\mathrm{out}} = RP$, with $R = 5$ jointly trained rules and $P = 7$ output bits per rule.

Each input sample contains two vectors, $\mathbf{x}$ and $\mathbf{y}$, concatenated into a shared input representation. The model is trained on rule indices 0–4, displayed in the paper as rules 1–5. Each rule defines a distinct transformation of the same input pair and produces an output vector of length P. The rule-specific outputs are concatenated and predicted by a shared network.

All tasks are introduced at the beginning of training and optimized jointly. We do not use curriculum learning, sequential task training, or task-specific pretraining. The training objective is

$$\mathcal{L} = \frac{1}{R}\sum_{r=1}^{R} W_r \mathcal{L}_r + l_{\mathrm{train}}\Omega_\gamma(\theta), \qquad \Omega_\gamma(\theta) = \sum_{B\in\mathcal{B}} \frac{1}{|B|}\sum_{q\in B}(|\theta_q| + \varepsilon)^\gamma,$$

where $\mathcal{L}_r$ is the mean-squared error averaged over samples and the P outputs of rule r, $W_r = 1$ by default, $\mathcal{B}$ is the set of trainable non-bias weight matrices, and $|B|$ is the number of scalar weights in matrix B. The implementation uses $\varepsilon = 10^{-8}$ and sums the per-matrix means rather than an unnormalized sum over all weights. Biases are excluded from the penalty. The task-weighting experiment sets one W_r to 5 while retaining the factor $1/R$ and the same l_{train} (Supplementary Note S7).

Robustness-simulation setup.

- Optimizer: Adam with learning rate 0.001.
- Batch size: 128.
- Hidden layers: three layers with 300 units per layer.
- Dataset size: 5,000 samples with a 50/50 training–validation split.
- Independent repeats: 30, starting from seed 300.
- Number of jointly trained tasks: 5 in the default experiments.
- Regularization exponent: $\gamma = 1$ in the robustness simulations; $\gamma \in \{0.5, 1, 2\}$ in the regularizer-form comparison.
- Sparsity coefficient: $l_1 = 5.0$ for MLP robustness simulations.

Training configurations by experiment. Saved training configurations distinguish the experiments listed below. All listed configurations use Adam, 5,000 training epochs, rules 1–5 (implementation indices 0–4), and a checkpoint interval of 1,000 epochs. Each experimental condition has 30 analyzed independent runs, except the TinyTransformer robustness analysis, which includes 23. The sweep contains 21 conditions; the task-weighting experiment contains six. The four mother-network conditions used for recomposition are drawn from the sweep and inherit its training settings.

Experiment	Learning rate	Training batch size
MLP robustness: binary, logic and MNIST	10^{-3}	128
TinyTransformer robustness	10^{-3}	128
Sparsity-strength/exponent sweep (Figs. 3, 4)	10^{-4}	512
Task weighting (Fig. 5)	5×10^{-2}	128
Recomposition mother networks (Fig. 6b,c)	10^{-4}	512

The TinyTransformer analysis uses the 23 saved runs trained with batch size 128. The seven runs trained with batch size 500 are excluded to keep the training batch size uniform; the original training records and analysis exports are retained. Training batch sizes in this table are distinct from the analysis batch size of 64 specified in the reproduction protocol. These values summarize saved configurations rather than software defaults; settings from one experiment are not transferred to another.

S2.2 TinyTransformer

The TinyTransformer provides a lightweight non-MLP architecture for testing whether functional-token extraction depends on fully connected hidden layers. Each paired binary input is split into left and right halves, and both halves are projected into a shared 32-dimensional token space. The resulting two-token sequence is processed by two encoder-style self-attention blocks with feedforward width 64. The final representation is obtained by mean-pooling the two tokens and passing the pooled vector through a linear prediction head with RP outputs.

In the robustness simulation, the TinyTransformer is trained on the same Binary Sequence Calculation rules as the MLP, using rule indices 0–4. The encoder is trainable (`train_encoder=True`), so both the token encoder and output head are optimized jointly. Functional tokens are constructed from scalar weight coordinates in the trained parameter matrices, profiled and retained with the same definitions used for the MLP experiments.

Robustness-simulation setup.

- Optimizer: Adam with learning rate 0.001.
- Training batch size: 128; 23 independent runs included in the analysis.
- Token/hidden dimension: 32.
- Transformer encoder blocks: two.
- Feedforward hidden dimension: 64.
- Dataset size: 5,000 binary-pair samples with a 50/50 training–validation split.
- Number of jointly trained arithmetic tasks: 5.
- Regularization exponent: $\gamma = 1$.
- Sparsity coefficient: $l_1 = 0.5$ for the transformer robustness simulation.

S3 Ablation and Taylor Profiling

All functional-token analyses in the main text use scalar weight coordinates as tokens. For a trained model with parameters θ and rule-specific output block f_r, ablating a weight token q means replacing θ_q by zero while leaving all other parameters fixed. The ablation importance score is

$$S^{\mathrm{abl}}_{r,q} = \mathbb{E}_{x\sim\mathcal{D}_r} \left\| f_r(x;\theta) - f_r(x;\theta_{-q}) \right\|_2^2, \tag{S.59}$$

where θ_{-q} denotes the parameter vector after zeroing coordinate q.

As a complementary first-order approximation, we estimate the squared output sensitivity of each scalar weight:

$$S^{\mathrm{tay}}_{r,q} = \theta_q^2 \, \mathbb{E}_x \left\| \frac{\partial f_r(x;\theta)}{\partial \theta_q} \right\|_2^2. \tag{S.60}$$

This approximates the squared first-order output perturbation associated with removing a scalar weight; it is not a task-loss gradient attribution. For each batch B, rule r and probe $k = 1, \ldots, K$, independent Rademacher signs $z^{B,k,r}_{i,d} \in \{-1, +1\}$ are drawn for each sample i and output coordinate d. The estimator is

$$g_{B,k,r,q} = \frac{\partial}{\partial \theta_q} \sum_{i\in B} \sum_{d=1}^{P} z^{B,k,r}_{i,d} f_{r,d}(x_i;\theta), \qquad \widehat{S}_{r,q} = \frac{1}{NK} \sum_{B} \sum_{k=1}^{K} (\theta_q g_{B,k,r,q})^2. \tag{S.61}$$

Here N is the total number of profiled samples. The expectation over the probes equals $\theta_q^2 N^{-1} \sum_i \|\partial f_r(x_i)/\partial \theta_q\|_2^2$. There is no additional division by the output dimension P. The confirmed Taylor runs in Figs. 2c,d,f and 4a used $K = 4$; this finite probe count does not itself guarantee a small uniform estimation error.

For multiplicative gates m_q, $\partial f_r/\partial m_q|_{m=1} = \theta_q \partial f_r/\partial \theta_q$; thus this scalar-weight score corresponds to the squared gate-Jacobian quantity in the local theoretical model.

S3.1 Token-Extraction and Modularity Settings

The extraction quantile is fixed at $\tau = 0.75$. For each trained network n, profiled weight matrix b and rule r, the numerical threshold is

$$T_{n,b,r} = \mathrm{Quantile}_{0.75}\{S_{n,b,r,q} : q \in \mathcal{C}_{n,b},\ S_{n,b,r,q} \text{ finite}\}, \tag{S.62}$$

where $\mathcal{C}_{n,b}$ is the set of profiled candidate coordinates. The important mask is

$$I_{n,b,r,q} = \mathbf{1}\{S_{n,b,r,q} \text{ finite}\}\mathbf{1}\{S_{n,b,r,q} > \epsilon_S\}\mathbf{1}\{S_{n,b,r,q} \geq T_{n,b,r}\}. \tag{S.63}$$

The quantile is computed before score-floor filtering, so finite zero scores enter its calculation. The numerical threshold is recomputed for every network, matrix and rule, and at every profiled training snapshot. Fixed τ is not a fixed numerical score threshold. Legacy configuration fields `cutoff_mode=fixed`, `panel_a_threshold` and `score_ratio_threshold` do not set the threshold in this `score_quantile` branch. If a candidate cap is supplied, the implementation profiles at most that many coordinates per matrix, chosen by absolute weight magnitude; unprofiled coordinates are not selected. For a rule set C, masks are combined by union,

$$I_{C,b,q} = \max_{r \in C} I_{r,b,q}. \tag{S.64}$$

We construct a mask separately for each profiled weight matrix and apply all masks simultaneously, including the output weights. Biases and other parameters outside the masking set remain unchanged. This subnetwork intervention differs from zeroing one scalar weight at a time to compute its importance. The retained-weight ratio is

$$\rho_{\mathrm{mask}}(C) = \frac{\sum_b \sum_{q \in W_b} I_{C,b,q}}{\sum_b |W_b|}. \tag{S.65}$$

Its numerator counts mask-selected positions, including selected coordinates with zero weight; its denominator counts all scalar coordinates in the analyzed weight matrices, including zero-valued entries and excluding biases. It is not the nonzero fraction of the full network.

Analysis configuration and reproduction checks. The saved summaries for Figs. 1c–f and 2a,b,d,f record $\tau = 0.75$, $\epsilon_S = 10^{-9}$, $\epsilon_{\mathrm{out}} = 10^{-8}$ and no candidate cap; the score floor is also present in all 150 original analysis exports. The current figures use 143 of these exports: 120 MLP exports and 23 TinyTransformer exports. Seven TinyTransformer runs trained with batch size 500 are excluded from Fig. 2f, leaving the 23 runs trained with batch size 128; all original exports are preserved. The Taylor groups record four probes.

For reproducibility, we specify the current analysis configuration for Figs. 3 and 4a: $\tau = 0.75$, $\epsilon_S = 10^{-9}$, all scalar-weight candidates, all available test samples and an analysis batch size of 64. Fig. 3 uses ablation profiling with $\epsilon_{\mathrm{out}} = 10^{-24}$; Fig. 4a uses randomized Taylor profiling with $\epsilon_{\mathrm{out}} = 10^{-8}$, four probes and seed 0. The verified execution devices are CUDA for ablation and CPU for Taylor; fixing the Taylor device preserves the random-probe sequence. These settings define the adopted reproduction protocol.

The Fig. 3c,d distributions match the saved importance-weighted NIP results and implement Eq. (13); rule-count purity is a different statistic and was not used for these panels. Recomputing six representative ablation checkpoints with the adopted configuration gave a maximum absolute difference in M of 1.04×10^{-7}, identical active-token counts, and identical single-rule relative accuracies and retained-weight ratios. Recomputing all 600 Taylor checkpoints underlying Fig. 4a gave a maximum absolute difference in M of 1.60×10^{-8} and identical active-token counts. These checks reassess saved trained models rather than retrain them.

For Fig. 5, all 180 newly trained 300-unit models underwent fresh scalar-weight ablation profiling. Fig. 6b,c uses fresh profiling of all 120 saved 300-unit mother models, and Fig. 6d and Supplementary Fig. S1 use fresh profiling of three saved auxiliary joint models. All these analyses use $\tau = 0.75$, $\epsilon_S = 10^{-9}$, $\epsilon_{\mathrm{out}} = 10^{-8}$, perturbation batch size 64 and no candidate cap. All 194,700 scalar weights, including the output matrix, are candidates; masks are constructed separately per matrix and applied simultaneously, while biases remain unchanged. Fig. 5 uses $l_{\mathrm{train}} = 5$ and $\gamma = 1$, without display rescaling. The recomposition conditions retain the sweep's experiment/display strength mapping. The saved 10^{-6} score-ratio field is not a selection threshold in the `score_quantile` branch, and probe-count metadata in ablation exports do not imply Taylor profiling.

Training regularization strength l_{train}, extraction quantile τ, realized score threshold T, score floor ϵ_S and output-energy constant ϵ_{out} have distinct roles. In the saved Lasso/exponent sweep used for Figs. 3 and 4, the displayed strength is $l = 5l_{\text{train}}$. Thus displayed $l = 0, 1, 5, 10$ corresponds to $l_{\text{train}} = 0, 0.2, 1, 2$. This mapping is not assumed for experiments lacking corresponding historical records. Experiment-specific training settings are provided in Supplementary Note S2; task-weighting and recomposition/cost protocols are detailed in Supplementary Notes S7 and S8, respectively. Rule accuracy is mean agreement over samples and output bits after thresholding predictions at > 0.5, not exact-vector accuracy; relative accuracy divides this measure by that of the full network. In the main-text definition of M, active tokens satisfy $a_q > \epsilon_M = 10^{-12}$, distinct from the raw-score floor ϵ_S; M is undefined if no token is active.

S4 Implementation Details

Token-level analyses are performed from saved PyTorch activations, model weights, and validation data, with MATLAB-compatible exports retained only for figure-generation compatibility.

- Export activations, outputs, task labels, and non-bias trainable weight matrices from trained models.
- Compute ablation scores $S^{\text{abl}}_{r,q}$ or Taylor scores $S^{\text{tay}}_{r,q}$ for scalar weight tokens.
- Convert scores into rule-wise important masks using the fixed score-quantile rule in Eq. 6.
- Construct each retained rule combination by taking the union of scalar weight tokens important to any selected rule.
- Construct the mask separately for each profiled matrix, then simultaneously zero unselected coordinates across all profiled matrices, including output weights; biases and other unmasked parameters remain unchanged.
- Compute relative accuracy matrices for single-rule functional tokens and multi-rule token combinations.

S5 Task Family Definitions

This section summarizes the task families used in the robustness simulations. The implementation uses zero-based rule indices, whereas the paper displays the same rules as rule 1, rule 2, and so on. The default active rule set is indices 0–4, corresponding to rules 1–5.

S5.1 Binary Sequence Calculation

This task family applies arithmetic rules to pairs of binary input patterns.

Input. Two binary vectors $x, y \in \{0, 1\}^P$ are sampled as paired binary patterns. In the robustness simulation, $P = 7$, 5,000 unique pairs are sampled without replacement from the $2^P \times 2^P$ possible pairs, and the model receives the concatenated vector $[x; y] \in \{0, 1\}^{2P}$.

Output. Each selected rule produces a binary vector of length P. With R selected rules, the outputs are concatenated into

$$Y \in \{0, 1\}^{RP}.$$

Rule Set. Let $a = \text{bin2dec}(x)$ and $b = \text{bin2dec}(y)$; a and b are the arithmetic operands used in the main text and figures, whereas x and y denote their binary encodings. Arithmetic rules are applied in integer space and then mapped back to binary using dec2bin, retaining only the last P bits:

1. Addition: $a + b$
2. Multiplication: ab
3. Square of a: a^2
4. Square of b: b^2
5. Mixed multiplication: $a(a + b)$
6. Alternative interaction: $b(a + b)$

S5.2 Sparse Pattern Logic

This task family applies elementwise logical operations to sparse binary patterns.

Input. Two binary vectors $x, y \in \{0,1\}^P$ are sampled independently from a sparse Bernoulli distribution. In the robustness simulation, $P = 7$ and the sparsity parameter is 0.7, so each bit is zero with probability 0.7 and one with probability 0.3. The model receives the concatenated vector $[x; y] \in \{0,1\}^{2P}$. Sample rows are shuffled and split without deduplicating pattern pairs across the training and held-out sets. Each of the 30 saved runs contains 2,500 held-out pairs.

Output. Each selected rule produces a binary vector in $\{0,1\}^P$. With R selected rules, the final output is

$$Y \in \{0,1\}^{RP}.$$

Rule Set. Logical operations are applied elementwise:

1. AND: $x \wedge y$
2. OR: $x \vee y$
3. XOR: $x \oplus y$
4. NOT of the first input: $\neg x$
5. Identity of the first input: x
6. Identity of the second input: y

S5.3 MNIST Digit-Pair Arithmetic

This task family applies arithmetic rules to MNIST digit labels while presenting the corresponding image pairs as input.

Input. Two MNIST images I_1, I_2 with digit labels $d_1, d_2 \in \{0, \ldots, 9\}$ are selected. In the robustness simulation, digit pairs are balanced over the 100 possible label pairs, and image instances for each digit are sampled from the MNIST training split. Images are normalized, flattened, and concatenated into a single input vector. The split is performed on generated image-pair rows rather than on unique source images, so the same source image can occur in both sets. Each of the 30 saved runs contains 2,500 held-out pairs.

Output. Rules operate on integer labels d_1, d_2. Each rule output is converted to binary and truncated to the last $P = 7$ bits. With R selected rules, the output is

$$Y \in \{0,1\}^{RP}.$$

Rule Set.

1. Addition: $d_1 + d_2$
2. Multiplication: $d_1 d_2$
3. Square of d_1: d_1^2
4. Square of d_2: d_2^2
5. Mixed multiplication: $d_1(d_1 + d_2)$
6. Alternative interaction: $d_2(d_1 + d_2)$

S6 Representational Analysis

We characterized the internal representations of the trained networks using three complementary measures: cross-layer representational similarity, within-layer spectral complexity, and pairwise hidden-unit correlation.

Cross-layer representational similarity. We quantified representation similarity across layers using linear centered kernel alignment (CKA). For each layer ℓ, hidden-unit activations were collected across the same set of input samples and arranged into an activation matrix $\mathbf{X}^{(\ell)}$. After centering the activation matrices across samples, linear CKA between layers ℓ and m was computed as

$$\mathrm{CKA}(\ell, m) = \frac{\left\| \tilde{\mathbf{X}}^{(\ell)\top} \tilde{\mathbf{X}}^{(m)} \right\|_F^2}{\left\| \tilde{\mathbf{X}}^{(\ell)\top} \tilde{\mathbf{X}}^{(\ell)} \right\|_F \left\| \tilde{\mathbf{X}}^{(m)\top} \tilde{\mathbf{X}}^{(m)} \right\|_F}.$$

Higher CKA values indicate greater similarity between the representational geometries of two layers.

PCA spectral entropy. To characterize the complexity of the representation within each layer, we performed principal component analysis on its activation matrix. Let $\lambda_i^{(\ell)}$ denote the fraction of variance explained by the i-th principal component of layer ℓ, normalized such that

$$\sum_i \lambda_i^{(\ell)} = 1.$$

The PCA spectral entropy was then defined as

$$\mathcal{H}^{(\ell)} = -\sum_i \lambda_i^{(\ell)} \log \lambda_i^{(\ell)}.$$

A low spectral entropy indicates that the representation is concentrated in a small number of principal directions, whereas a high value indicates that variance is distributed more broadly across representational dimensions.

Pairwise hidden-unit correlation. To quantify statistical dependence among hidden units within each hidden layer, we computed pairwise Pearson correlations between their pre-activation responses. Let

$$\mathbf{H}^{(\ell)} = \left[\mathbf{h}_1^{(\ell)}, \ldots, \mathbf{h}_{d_\ell}^{(\ell)} \right] \in \mathbb{R}^{N \times d_\ell}$$

denote the pre-activation response matrix of layer ℓ, where N is the number of input samples and d_ℓ is the number of hidden units. The correlation between hidden units i and j was calculated as

$$\rho_{ij}^{(\ell)} = \frac{\sum_{n=1}^{N} \left(h_{ni}^{(\ell)} - \bar{h}_i^{(\ell)} \right) \left(h_{nj}^{(\ell)} - \bar{h}_j^{(\ell)} \right)}{\sqrt{\sum_{n=1}^{N} \left(h_{ni}^{(\ell)} - \bar{h}_i^{(\ell)} \right)^2} \sqrt{\sum_{n=1}^{N} \left(h_{nj}^{(\ell)} - \bar{h}_j^{(\ell)} \right)^2}}.$$

We excluded the diagonal entries and retained one value for each unordered hidden-unit pair by extracting the upper triangular portion of the correlation matrix. Because both positive and negative correlations indicate statistical coupling, we analyzed the absolute correlation magnitude,

$$c_{ij}^{(\ell)} = \left| \rho_{ij}^{(\ell)} \right|.$$

The values were grouped into 100 equally spaced bins over $[0, 1]$, and the histogram was normalized to represent the probability distribution of pairwise correlations within each layer. Distributions concentrated near zero indicate weakly correlated and more diverse hidden-unit responses, whereas greater mass at high correlation values indicates stronger coordination or redundancy among hidden units.

S7 Task-Weighting Functional Allocation Analysis

This section defines the analysis in Fig. 5, generated by `example/fig56_protocol.py` and plotted with `example/figure6.py`. All six conditions use the same MLP architecture with three hidden layers of 300 units. Each condition contains $K = 30$ runs with seeds 300–329, giving 180 newly trained networks. Training uses 5,000

epochs, Adam with learning rate 0.05, batch size 128, $l_{\text{train}} = 5$ and $\gamma = 1$. Each seed generates 5,000 unique binary input pairs, split equally into training and held-out sets and shared across conditions. The model initialization and optimizer random seeds are the run seed plus 100 and 200, respectively. Figure labels use rules 1–5; the implementation indexes them as 0–4.

Experimental design. Let $c = 0$ denote the equal-weight control, for which $W_r^{(0)} = 1$ for every rule. In intervention condition $c = u$, rule u receives five-fold loss weight:

$$W_r^{(u)} = \begin{cases} 5, & r = u, \\ 1, & r \neq u. \end{cases} \tag{S.66}$$

Functional tokens are extracted independently from each trained network by ablation profiling and the fixed rule-wise score-quantile mask used in the main analysis. Importance profiling and evaluation of the masked subnetworks use the same held-out set, as in the robustness and recomposition analyses; reported retention therefore measures preservation on that analysis set, rather than generalization to a second untouched evaluation set.

Condition-level matrices. Let $A^{\text{full}}_{c,r,k}$ be the unmasked-network accuracy in condition c, evaluated on rule r in run k, and let $\rho_{c,r,k}$ be the retained scalar weight fraction of the functional token defined for rule r. Bars denote averages over runs. Panels a and b report

$$\Delta A_{u,r} = \overline{A}^{\text{full}}_{u,r} - \overline{A}^{\text{full}}_{0,r}, \qquad \Delta\rho_{u,r} = \overline{\rho}_{u,r} - \overline{\rho}_{0,r}, \tag{S.67}$$

for every upweighted rule u and evaluated/token-defining rule r; panel b displays $10^4\Delta\rho_{u,r}$.

To define panel c, let $A^{\text{ret}}_{c,s|r,k}$ be the relative accuracy on evaluated rule s when only the functional token defined for rule r is retained in run k. Its specialization margin is

$$m_{c,r,k} = A^{\text{ret}}_{c,r|r,k} - \frac{1}{R-1}\sum_{s\neq r} A^{\text{ret}}_{c,s|r,k}. \tag{S.68}$$

Panel c shows $\overline{m}_{u,r}$ for each upweighted condition and rule-defined token. Unlike panels a and b, this panel reports the margin in the intervention condition itself rather than a control-subtracted change.

Target-rule summaries and paired trials. Panel d uses the five diagonal condition-level pairs $(\Delta A_{u,u}, 10^4\Delta\rho_{u,u})$. For a metric X (accuracy or retained-weight ratio), the implemented marginal error is $e_{c,X} = \sigma_{c,X}/\sqrt{K}$, where $\sigma^2_{c,X} = K^{-1}\sum_k (X_{c,k} - \overline{X}_c)^2$ (population standard deviation, `ddof=0`). The displayed error for a difference is $e_{\Delta X} = \sqrt{e^2_{u,X} + e^2_{0,X}}$; the weight-ratio error is multiplied by 10^4 on the plotted axis. This retains the original error-bar convention and is not the SEM of paired differences, because it omits their covariance. The displayed Pearson coefficient is descriptive across the five rule means.

For panel e, upweighted and control runs are matched by repeat index. The target-rule differences are

$$\delta A_{u,k} = A^{\text{full}}_{u,u,k} - A^{\text{full}}_{0,u,k}, \qquad \delta\rho_{u,k} = \rho_{u,u,k} - \rho_{0,u,k}. \tag{S.69}$$

Pooling the five rules gives $RK = 150$ matched pairs. The figure reports their Pearson correlation ($r = 0.7835$), nominal two-sided p value ($p = 2.18 \times 10^{-32}$), and least-squares fit. Conditions share run seeds and the same control networks; this pooled p value is not adjusted for clustering by seed, shared controls, or task identity. It therefore does not establish an independent association after accounting for that dependence. These associations quantify covariation between performance and selected support under task weighting, without identifying either quantity as the cause of the other.

S8 Compositional Reconstruction of Functional Tokens

S8.1 Source networks and component functional tokens

For each sparsity condition, a mother MLP was first trained jointly on rules 1–5 of the Binary Sequence Calculation task family under L_γ regularization, with $\gamma = 1$. The sparsity-strength parameter l therefore denotes the Lasso strength used during training of the mother network, rather than a regularization term applied during recomposition.

For each of four displayed regularization strengths and 30 independent trials, the saved 300-unit mother model was profiled again using scalar-weight ablation and the masking procedure described above. The 120 fresh analyses

produced masks for addition, multiplication, square and composite rule 5. Each mask and its learned parameters were then held fixed while evaluating the direct rule-5 control and the two recomposition pathways, giving 360 evaluations. Recomposition does not retrain or fine-tune the components, and it does not merge their masks.

Each functional token retained the architecture and learned parameter values inherited from its mother network, while parameters excluded by its functional mask were set to zero. The masks of different functional tokens were not combined through a union of scalar weight coordinates. Instead, each token remained a separate masked MLP, and recomposition was implemented by explicitly connecting the output of one token to the input of another.

No parameters were retrained or fine-tuned during this procedure, and no additional L_1 penalty or other optimization objective was applied after the component tokens were connected. The resulting reconstructions therefore constitute zero-shot compositions of fixed functional subnetworks.

S8.2 Notation and output selection

Let the original input be the concatenation of two binary vectors,

$$\mathbf{u} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}, \qquad \mathbf{x}, \mathbf{y} \in \{0,1\}^7.$$

The corresponding integer operands are $a = \mathrm{bin2dec}(\mathbf{x})$ and $b = \mathrm{bin2dec}(\mathbf{y})$, using the original task encoding.

For a functional token F, its masked MLP was evaluated using the same feedforward computation as the original network. Unselected scalar weights in all profiled matrices remained zero throughout the forward pass; biases and other unmasked parameters retained their trained values.

The output of the original multitask MLP consists of consecutive seven-dimensional blocks corresponding to different rules. In the implementation, dimensions 1–7 correspond to addition, dimensions 8–14 to multiplication, dimensions 15–21 to the square rule used to compute a^2, and dimensions 29–35 to rule 5.

Intermediate outputs passed between functional tokens were binarized using

$$\mathcal{B}(\mathbf{z}) = \mathbb{I}[\mathbf{z} > 0.5],$$

where the threshold was applied element-wise.

S8.3 Type 1: sequential addition and multiplication

The type 1 reconstruction implements

$$a(a+b)$$

as a sequential composition of the addition and multiplication functional tokens.

The addition token was first evaluated on the original input:

$$\mathbf{o}_{\mathrm{add}} = F_{\mathrm{add}}(\mathbf{u}).$$

The seven-dimensional addition output was selected from dimensions 1–7 and binarized:

$$\hat{\mathbf{z}}_{\mathrm{add}} = \mathcal{B}\left(\mathbf{o}_{\mathrm{add}}^{(1:7)}\right).$$

This intermediate output was concatenated with the original vector $\mathbf{x}$:

$$\mathbf{u}_{\mathrm{T1}} = \begin{bmatrix} \hat{\mathbf{z}}_{\mathrm{add}} \\ \mathbf{x} \end{bmatrix}.$$

The resulting vector was supplied to the multiplication functional token:

$$\mathbf{o}_{\mathrm{T1}} = F_{\mathrm{mult}}\left(\mathbf{u}_{\mathrm{T1}}\right).$$

Because the final component is the multiplication token, dimensions 8–14 of $\mathbf{o}_{\mathrm{T1}}$ were used as the reconstructed rule-5 output. The pathway therefore implements

$$(a+b)a = a(a+b).$$

The addition and multiplication tokens retained their independently extracted masks and parameter values throughout the reconstruction.

S8.4 Type 2: parallel square and multiplication followed by addition

The type 2 reconstruction uses the identity

$$a(a+b) = a^2 + ab.$$

The square and multiplication functional tokens were first evaluated independently on the original input:

$$\mathbf{o}_{\mathrm{sq}} = F_{\mathrm{sq}}(\mathbf{u}), \qquad \mathbf{o}_{\mathrm{mult}} = F_{\mathrm{mult}}(\mathbf{u}).$$

The square-rule output corresponding to a^2 was selected from dimensions 15–21, whereas the multiplication output corresponding to ab was selected from dimensions 8–14. Both intermediate outputs were binarized:

$$\hat{\mathbf{z}}_{\mathrm{sq}} = \mathcal{B}\left(\mathbf{o}_{\mathrm{sq}}^{(15:21)}\right),$$

$$\hat{\mathbf{z}}_{\mathrm{mult}} = \mathcal{B}\left(\mathbf{o}_{\mathrm{mult}}^{(8:14)}\right).$$

The two intermediate vectors were concatenated:

$$\mathbf{u}_{\mathrm{T2}} = \begin{bmatrix} \hat{\mathbf{z}}_{\mathrm{sq}} \\ \hat{\mathbf{z}}_{\mathrm{mult}} \end{bmatrix},$$

and supplied to the addition functional token:

$$\mathbf{o}_{\mathrm{T2}} = F_{\mathrm{add}}\left(\mathbf{u}_{\mathrm{T2}}\right).$$

Because the final component is the addition token, dimensions 1–7 of $\mathbf{o}_{\mathrm{T2}}$ were used as the reconstructed rule-5 output. This pathway implements

$$a^2 + ab = a(a+b).$$

Thus, type 2 connects three fixed functional tokens in a parallel-then-sequential configuration without combining their parameter masks into a single network.

S8.5 Control conditions and accuracy evaluation

For each mother network, we evaluated three conditions:

1. the functional token extracted directly for rule 5, used as the non-recomposed control;
2. the type 1 reconstruction using addition followed by multiplication;
3. the type 2 reconstruction using square and multiplication followed by addition.

The corresponding full, unmasked multitask MLP was used as the performance reference. For both the full network and the directly extracted rule-5 token, output dimensions 29–35 were evaluated. For the type 1 reconstruction, dimensions 8–14 of the final multiplication token were evaluated, whereas dimensions 1–7 of the final addition token were evaluated for type 2.

All outputs were compared with the same seven-dimensional rule-5 target,

$$f_5(a,b) = a(a+b).$$

Final continuous predictions were binarized using a threshold of 0.5. For condition k, accuracy was calculated as the mean element-wise agreement between the binary prediction and the rule-5 target across all samples:

$$A_k = \frac{1}{7N} \sum_{n=1}^{N} \sum_{j=1}^{7} \mathbb{I}\left[\hat{b}_{nj}^{(k)} = y_{nj}^{(5)}\right],$$

where N is the number of evaluated samples.

Let A_{full} denote the rule-5 accuracy of the corresponding full mother network. Relative reconstruction accuracy was defined as

$$A_{\mathrm{rel}}^{(k)} = \frac{A_k}{\max(A_{\mathrm{full}}, \varepsilon)}, \qquad \varepsilon = 10^{-8}.$$

The numerical guard matches the implementation. Across the 360 saved evaluations, $\min A_{\mathrm{full}} = 0.782$ and $\min s_5 \approx 0.01126$, so neither denominator guard is active. The numerator and denominator are always evaluated on rule 5. The accuracies of the addition, multiplication, and square component tasks are not independently included in this metric.

S8.6 Parameter-normalized reconstruction score

For an extracted functional token, the retained-weight fraction was calculated as

$$s = \frac{N_{\mathrm{nz}}}{N_{\mathrm{total}}},$$

where N_{nz} is the number of nonzero scalar-weight entries and N_{total} is the total number of scalar-weight entries in the corresponding masked model. In the figure-generation implementation, these quantities are summed across weight matrices; biases are excluded. The cached retained-parameter field is an alias of the retained-weight fraction.

For the directly extracted rule-5 functional token, this fraction was

$$s_5 = \frac{N_{\mathrm{nz},5}}{N_{\mathrm{total},5}}.$$

The parameter-normalized reconstruction score was defined as

$$E_w^{(k)} = \frac{A_{\mathrm{rel}}^{(k)}}{\max(s_5, \varepsilon)}.$$

Here $s_5 = \rho_{\mathrm{nz},5}(l)$ in the main-text notation, and $A_{\mathrm{rel}}^{(k)} = A_{C,5}$ for configuration $C = k$. The same value of s_5 was used as the denominator for the directly extracted rule-5 control and for both reconstructed pathways. Consequently, $E_w^{(k)}$ provides a common parameter-normalized comparison of rule-5 reconstruction performance. It should not be interpreted as the pathway-specific parameter efficiency of the type 1 or type 2 reconstruction, because the actual retained-weight fractions of their component subnetworks are not used separately in this normalization.

Finally, the value of l associated with each reconstruction indicates the Lasso strength under which its mother network was trained. It does not describe sparsity introduced during recomposition, because the component functional tokens were connected and evaluated without further optimization.

S8.7 Auxiliary training and extraction cost

The auxiliary benchmark compares three jointly trained five-task models (seeds 830–832), those models followed by weight-token extraction, and 15 separately trained single-task controls (five per seed). Each network has three hidden layers of 300 units. This auxiliary experiment uses 600 epochs of SGD, learning rate 0.05, batch size 128, and 1,000 training plus 1,000 held-out samples; it is distinct from the 5,000-epoch Adam experiments. Both joint and single-task models used $l_{\mathrm{train}} = 1$, $\gamma = 1$, and a regularization offset of 10^{-8}. The trained models are reused, and weight profiling is recomputed with the protocol specified above.

For each run, the training proxy is

$$C_{\mathrm{train}} = N_{\mathrm{parameters}} N_{\mathrm{train}} N_{\mathrm{epochs}}.$$

For scalar-weight extraction, the implemented batching proxy is

$$C_{\mathrm{extract}} = N_{\mathrm{test}} \sum_b \lceil N_{\mathrm{weight},b} / B_{\mathrm{perturb}} \rceil,$$

where $N_{\mathrm{weight},b}$ counts scalar weights in matrix b and $B_{\mathrm{perturb}} = 64$. Each joint network has 194,700 candidate weights across four matrices. Profiling performs 194,700 single-weight perturbations in 3,045 batched perturbation-forward calls, each using 1,000 held-out samples. Thus there are 194,700,000 perturbed sample evaluations, whereas the batching proxy is 3,045,000; batching does not make those two counts equivalent. The unperturbed reference evaluation and subsequent masked-network evaluations are not included in this extraction proxy.

The training proxies are 1.17381×10^{11} for the joint network and 5.61621×10^{11} for the five separate networks combined. The benchmark adds the differently scaled training and extraction proxies to obtain a predefined heuristic total, $1.17384045 \times 10^{11}$ for joint training plus extraction. This sum is neither a FLOP count nor a wall-clock cost, and does not establish a hardware-independent or measured speed-up. Each run is normalized by its own joint-training-plus-extraction total; the separate-training ratio is 4.7845. Recomposition is excluded.

Fresh weight extraction took a mean of 13.42 seconds (sample SD 0.33 seconds) across the three runs on the recorded CUDA device. These measurements describe the new extraction stage only; they are not combined with historical training times. The mean full-network accuracies were 0.7641 for joint training and 0.8677 for separate training, so the benchmark is not an equal-accuracy comparison. Extracted tokens retained a mean relative accuracy of 0.9527 (s.e.m. 0.0047), averaging over the five rules within each joint model and then over the three models.

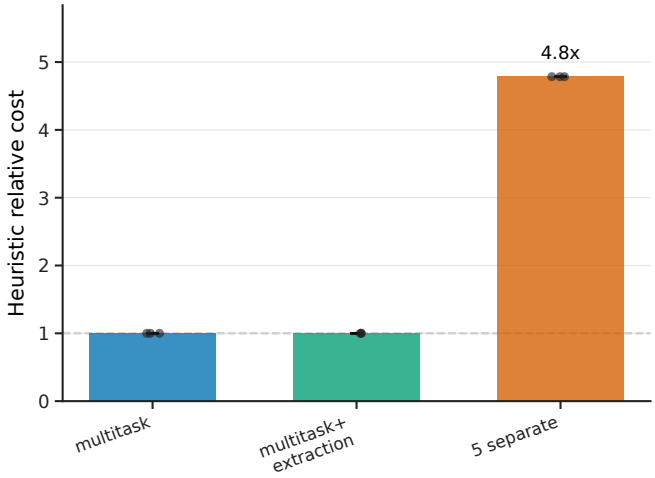


Figure S1: Auxiliary training and extraction accounting. Bars show means over three independent benchmark runs, error bars show s.e.m., and points show individual runs. The reference value of one is the jointly trained network plus extraction. The heuristic combines differently scaled proxies as specified in the text and excludes recomposition.